

Assignment 9.4

2303A51867

Batch-13

Task-1



```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

print("Addition:", add_numbers(10, 5))
print("Factorial:", factorial(5))
print("Is 7 Prime?", is_prime(7))
print("Is 8 Prime?", is_prime(8))
```

```
*** Addition: 15
    Factorial: 120
    Is 7 Prime? True
    Is 8 Prime? False
```

Prompt used:

Generate Google-style docstrings for the following Python functions. Include description, parameters with types, return value, and example usage.

Lab Assignment 9.4 | Assignment Steps HPC | HPC_ASSIGNMENT_3.ipynb - Colab | Untitled12.ipynb - Colab

colab.research.google.com/drive/18PpOHVtSsvipa2TpSF0MYo_ncGf5mX#scrollTo=ZuPczU08GeUe

Untitled12.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Examples:

```
>>> is_prime(7)
True
>>> is_prime(10)
False
>>> is_prime(1)
False
"""
if num <= 1:
    return False
for i in range(2, int(num**0.5) + 1):
    if num % i == 0:
        return False
return True

print("Addition:", add_numbers(10, 5))
print("Factorial:", factorial(5))
print("Is 7 Prime?", is_prime(7))
print("Is 8 Prime?", is_prime(8))
```

... Addition: 15
Factorial: 120
Is 7 Prime? True
Is 8 Prime? False

Variables Terminal

10:08 AM Python 3

Gemini

M Generate Google-style docstrings for the following Python functions. Include description, parameters with types, return values, and example usage.

I will regenerate the Google-style docstrings for the `add_numbers`, `factorial`, and `is_prime` functions in the existing cell, including descriptions, parameters, return values, and examples.

Explain the logic behind the 'factorial' function
Test 'is_prime' with negative numbers
Add a new function to sum a list of numbers

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution.
[Learn more](#)

Task 2:

def fibonacci(n):

```
seq = []
a, b = 0, 1

for _ in range(n):
    seq.append(a)
    a, b = b, a + b

return seq

print(fibonacci(10))
```

... [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Prompt used:

Insert inline comments only for complex or non-obvious logic. Avoid commenting trivial syntax.

The screenshot shows a Google Colab notebook titled 'Untitled12.ipynb'. The code cell contains a Python function `fibonacci(n)` that generates the first `n` Fibonacci numbers. The function is defined with inline comments: `a, b = 0, 1 # Initialize the first two Fibonacci numbers` and `a, b = b, a + b # Update a and b for the next Fibonacci number`. The function is called with `print(fibonacci(10))`, and the output is the list `[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]`.

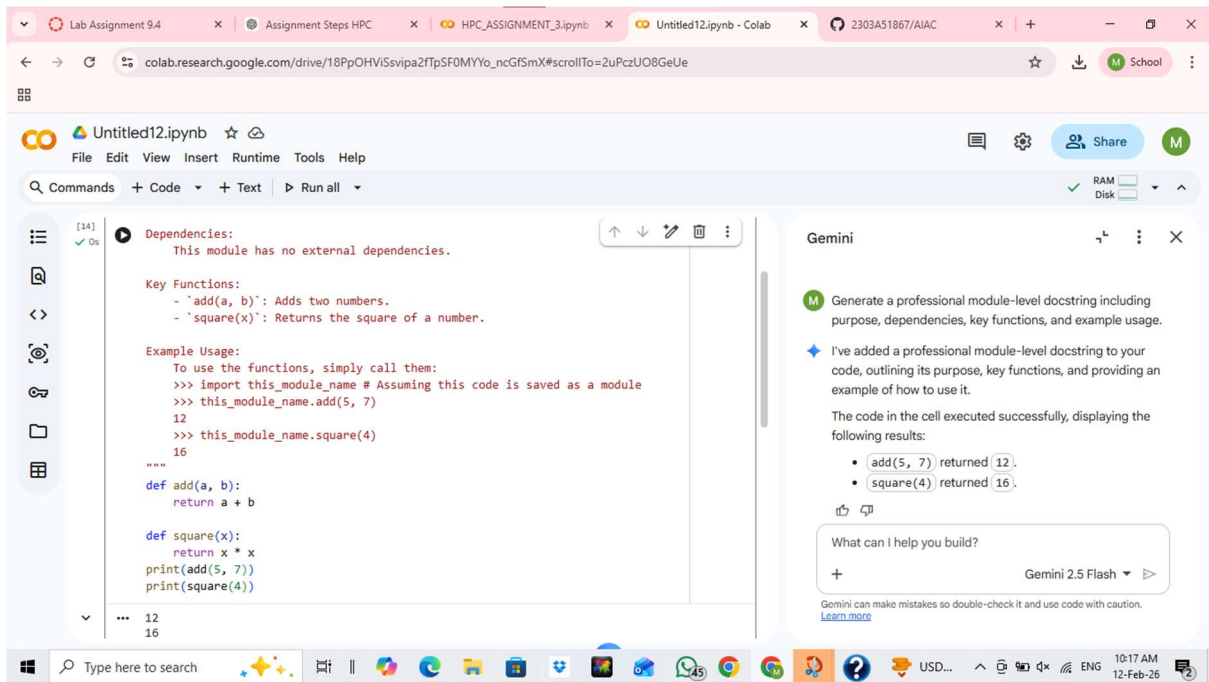
On the right side, there is a Gemini chat window. The chat history shows a prompt: 'Insert inline comments only for non-obvious logic. Avoid commenting simple syntax'. The response from Gemini is: 'I've added inline comments to the `fibonacci` function, highlighting the key parts of the logic. The `fibonacci` function executed successfully, printing the first 10 numbers in the sequence: `[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]`'. Below the response are several suggested prompts: 'Plot the generated Fibonacci sequence', 'Explain the recursive approach to generating Fibonacci numbers', and 'How to modify the function to generate Fibonacci numbers up to a certain limit?'. At the bottom of the chat window, there is a text input field with the placeholder 'What can I help you build?' and a 'Gemini 2.5 Flash' model selector.

Task3:

The screenshot shows a code editor with two Python functions. The first function is `def add(a, b):` which returns `a + b`. The second function is `def square(x):` which returns `x * x`. Below the functions, there are two print statements: `print(add(5, 7))` and `print(square(4))`. The output of the code is shown at the bottom: `12` and `16`.

Prompt used:

Generate a professional module-level docstring including purpose, dependencies, key functions, and example usage.



Task 4:

```
def square(x):
    return x * x
print(square(6))
```

... 36

Prompt used:

Convert the inline developer comments into structured Google-style docstrings and remove redundant comments.

Lab Assignment 9.4

Assignment Steps HPC

HPC_ASSIGNMENT_3.ipynb

Untitled12.ipynb - Colab

2303A51867/AIAC

colab.research.google.com/drive/18PpOHViSvipa2fTpSF0MYYo_ncGFsmX#scrollTo=2uPczUO8GeUe

Untitled12.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

171

0s

def square(x):
 """Calculates the square of a number.

 Args:
 x (int or float): The number to be squared.

 Returns:
 int or float: The square of the input number.

 Examples:
 >>> square(5)
 25
 >>> square(3.5)
 12.25
 """
 return x * x
 print(square(6))

36

Gemini

M

Convert the inline developer comments into structured Google-style docstrings and remove redundant comments

I've updated the `square` function with a Google-style docstring, including its purpose, parameters, return value, and an example.

Working...

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution.
[Learn more](#)

Variables Terminal

10:20 AM Python 3